Actividad 6

Sistema contador y decodificador

Índice

1. Introducción y objetivos 2
2. Creación del proyecto 2
3. Descripción VHDL del sistema global 3
4. Simulación del sistema 6
5. Asignación de pines de la FPGA 9
6. Configuración del contador en la FPGA 10
7. Recursos de la FPGA empleados 10
8. Esquemático estructural del circuito 11
9. Vista del circuito en la FPGA 13
Referencias 17

Actividad 6

Sistema contador y decodificador

1.- Introducción y objetivos

En esta práctica se va a proceder a implementar en la FPGA de la tarjeta de desarrollo el sistema conformado por el contador y el decodificador de 7 segmentos diseñados en la Actividad 4 y la Actividad 5.

La Figura 1 muestra la estructura que tiene el nuevo circuito. La cuenta del contador se debe mostrar a la salida en formato display de 7 segmentos y en formato binario.

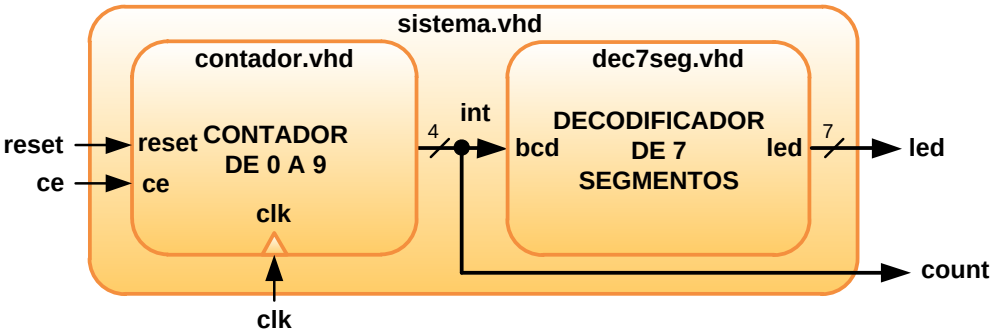


Figura 1. Esquema del sistema a diseñar.

Los objetivos a alcanzar en esta práctica son:

- Crear circuitos complejos basados en circuitos más sencillos
- Diseño jerárquico
- Analizar los recursos empleados en el diseño
- Implementación en la tarjeta de desarrollo del sistema conformado por el contador y el decodificador de 7 segmentos
- Ver el esquemático y extraer información de él
- Ver el circuito en el visor de FPGA y extraer información de él

2.- Creación del proyecto

Para la creación del proyecto de Vivado de esta actividad, deben seguirse los pasos descritos en el Apartado 3 de la Actividad 0. En este caso,

el nombre del proyecto es *Actividad6* (todo junto) y el resto de parámetros se deja igual.

En esta ocasión, en el apartado *Add Sources* del asistente para crear el nuevo proyecto, se debe clicar en *Add Files* (Figura 1), Con esto se abre una nueva ventana en la que incorporamos los ficheros VHDL de diseño y simulación al proyecto que se está creando.

Se debe incorporar los ficheros *dec7seg.vhd*, de la Actividad 4; y *ffD_reset.vhd*, *contador.vhd*, de la Actividad 5, según se muestra en la Figura 2. Los pasos a seguir se encuentran detallados en el Apartado 2 de la Actividad 3.

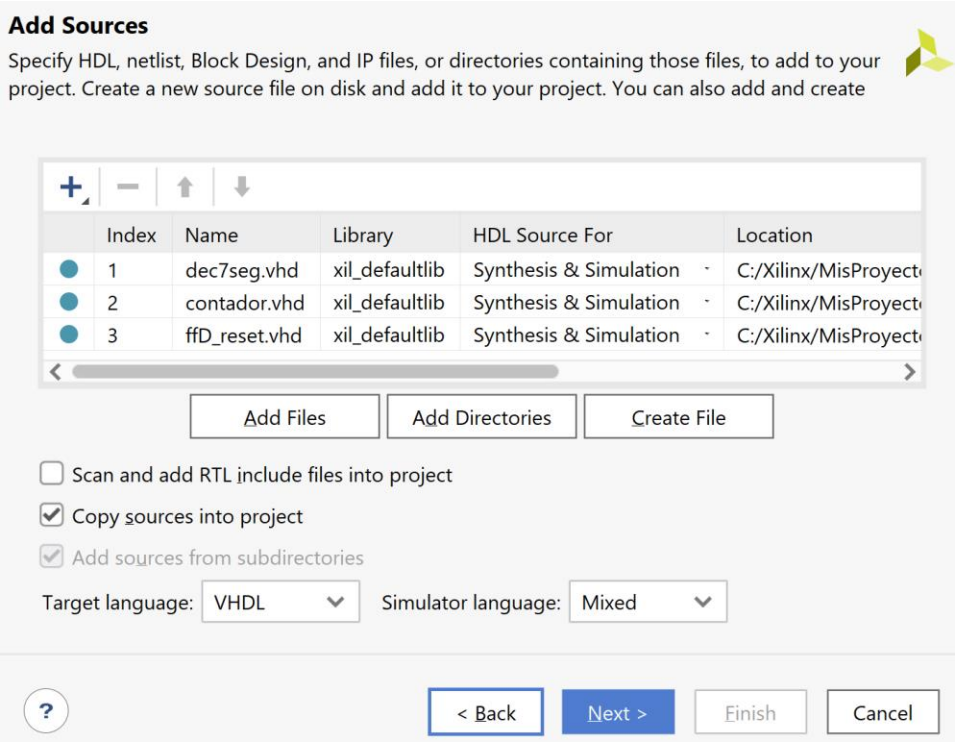


Figura 2. Captura de pantalla de *Add Sources*, con los ficheros a incorporar en el nuevo proyecto.

3.- Descripción VHDL del sistema global

El siguiente paso consiste en crear un código VHDL, denominado *sistema.vhd*, que englobe como subcircuitos al contador y al decodificador de 7 segmentos conforme a la Figura 1.

Para generar este código VHDL se procede de la siguiente manera:

1. Generar la plantilla del código VHDL del *top level* del sistema, *sistema.vhd*, a través del comando *New Source*, siguiendo los

pasos descritos en el apartado 3 de la Actividad 1, añadiendo en el asistente los puertos del componente (Figura 3).

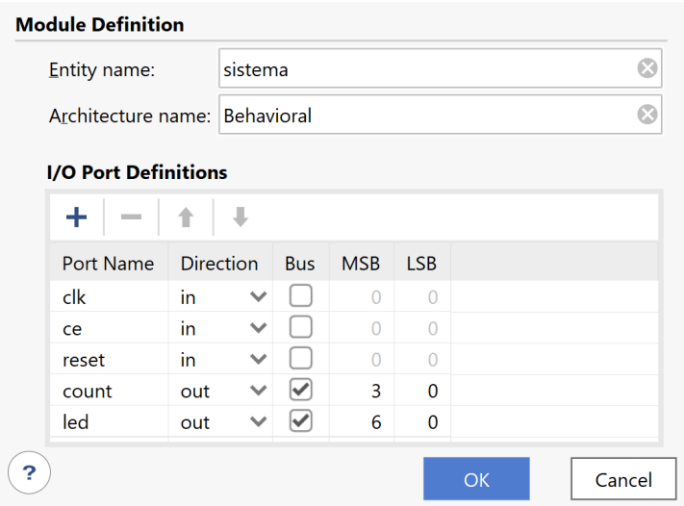


Figura 3. Definición de los puertos del *top level*.

2. En la plantilla generada, escribir el código en la arquitectura (Figura 4).

En la declaración de la arquitectura se encuentran:

- La declaración del componente *contador* (líneas 15-20).
- La declaración del componente *dec7seg* (líneas 22-25).
- La declaración de la señal interna *int* (línea 27). Esta señal interna sirve para conectar la salida del contador con la entrada del decodificador de 7 segmentos.

En parte funcional de la arquitectura se encuentran:

- El mapeo de puertos del subcircuito *contador* (líneas 31-35).
- El mapeo de puertos del subcircuito *dec7seg* (líneas 37-39).
- La asignación que conecta el puerto de salida *count* (definido en la entidad) con la salida del contador (la señal interna *int*).

Nótese que no se puede conectar la salida del contador directamente a la salida del sistema, puesto que esta conexión también entra en el puerto del decodificador, y no se pueden leer puertos de salida (confrontar con el Apartado 5 de la Actividad 1).

```
13 architecture Behavioral of sistema is
14
15     component contador
16         port (clk   : in std_logic;
17               reset : in std_logic;
18               ce    : in std_logic;
19               count : out std_logic_vector (3 downto 0));
20     end component;
21
22     component dec7seg
23         port (bcd : in std_logic_vector (3 downto 0);
24               led : out std_logic_vector (6 downto 0));
25     end component;
26
27     signal int : std_logic_vector (3 downto 0);
28
29 begin
30
31     Inst_contador : contador
32     port map (clk   => clk,
33               reset => reset,
34               ce    => ce,
35               count => int);
36
37     Inst_dec7seg : dec7seg
38     port map (bcd => int,
39               led => led);
40
41     count <= int;
42
43 end Behavioral;
```

Figura 4. Código VHDL de la arquitectura de *sistema.vhd*.

Una vez realizado el circuito del sistema, la ventana de jerarquía debe mostrar los 2 subcircuitos *dec7seg* y *contador* usados colgando del circuito *sistema*, como puede apreciarse en la Figura 5.

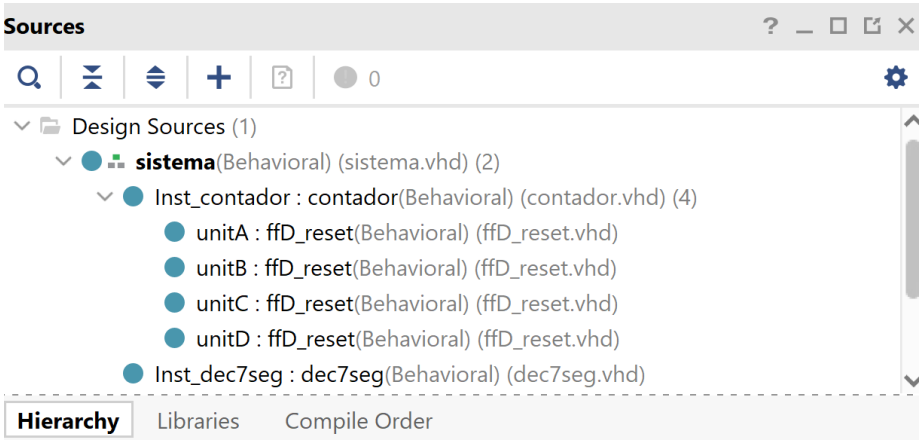


Figura 5. Captura de pantalla de la pestaña de jerarquía de *sistema*.

La sintaxis y el empleo de PORT MAP para conectar adecuadamente los subcircuitos se especifica en el Apartado 6 de la Actividad 1 y en el Apartado 4 de la Actividad 5.

Este es un ejemplo muy básico de un diseño jerárquico. En diseños complejos, lo normal es particionar un circuito muy complicado y descomponerlo en pequeñas partes que son fáciles de describir e implementar. A continuación, se van juntando estos subcircuitos como si piezas de un puzle se tratara. Es lo que se denomina una metodología de diseño *bottom-top*.

4.- Simulación del sistema

La simulación del sistema conformado por el contador y el decodificador de 7 segmentos requiere la creación del *test bench* correspondiente. Para la creación *test bench*, deben seguirse los pasos descritos en el Apartado 6 de la Actividad 1. En este caso, el nombre del *test bench* se puede poner *sistema_tb*. La arquitectura se muestra en la Figura 6.

La interpretación de este código es similar a la especificada en el Apartado 5 de la Actividad 5.

```
8  architecture Behavioral of sistema_tb is
9
10     component sistema
11         port (clk   : in std_logic;
12               ce    : in std_logic;
13               reset : in std_logic;
14               count : out std_logic_vector (3 downto 0);
15               led   : out std_logic_vector (6 downto 0));
16     end component;
17
18     signal clk   : std_logic := '0';
19     signal ce    : std_logic := '0';
20     signal reset : std_logic := '0';
21     signal count : std_logic_vector (3 downto 0);
22     signal led   : std_logic_vector (6 downto 0);
23
24 begin
25
26     dut : sistema
27     port map (clk   => clk,
28               ce    => ce,
29               reset => reset,
30               count => count,
31               led   => led);
32
33     -- Clock generation
34     clk <= not clk after 10 ns;
35
36     stimuli : process
37     begin
38         -- EDIT Adapt initialization as needed
39         ce <= '0'; reset <= '1';
40         wait for 100 ns;
41         reset <= '0';
42         wait for 100 ns;
43         ce <= '1';
44         wait;
45     end process;
46
47 end Behavioral;
```

Figura 6. Captura de pantalla de la arquitectura de *sistema_tb.vhd*.

La Figura 7 muestra cómo queda la ventana de jerarquía para el *test bench*. Se puede apreciar 4 niveles de jerarquía con todo desplegado en el panel.

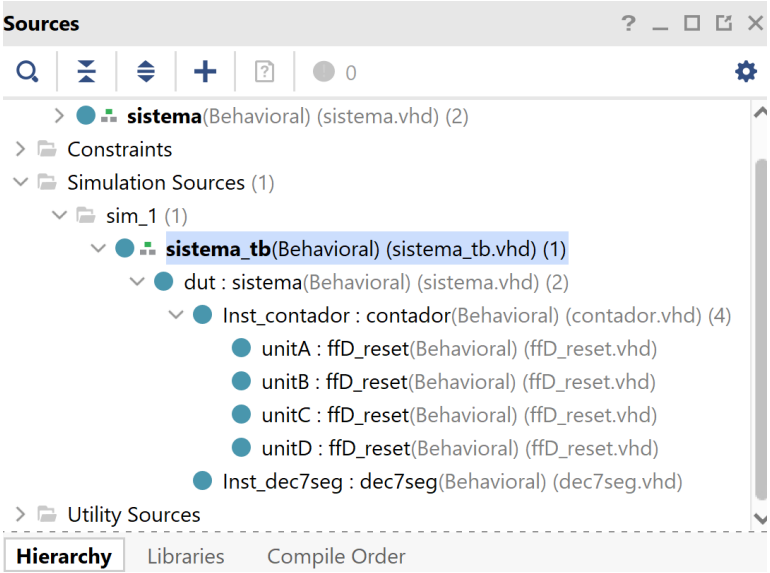


Figura 7. Captura de pantalla de la pestaña de jerarquía de *sistema_tb*.

La Figura 8 muestra el cronograma de simulación generado con el *test bench*. Se puede apreciar que el circuito se comporta adecuadamente, es decir, cuenta de 0 a 9 cíclicamente. Una vez llegado al 9, vuelve a 0, y así, indefinidamente mientras esté habilitado el contador y no se efectúe un reset. Además, la salida correspondiente al decodificador también es correcta, confrontando los resultados obtenidos en el cronograma con la Tabla 2 y la Figura 7 de la Actividad 4.

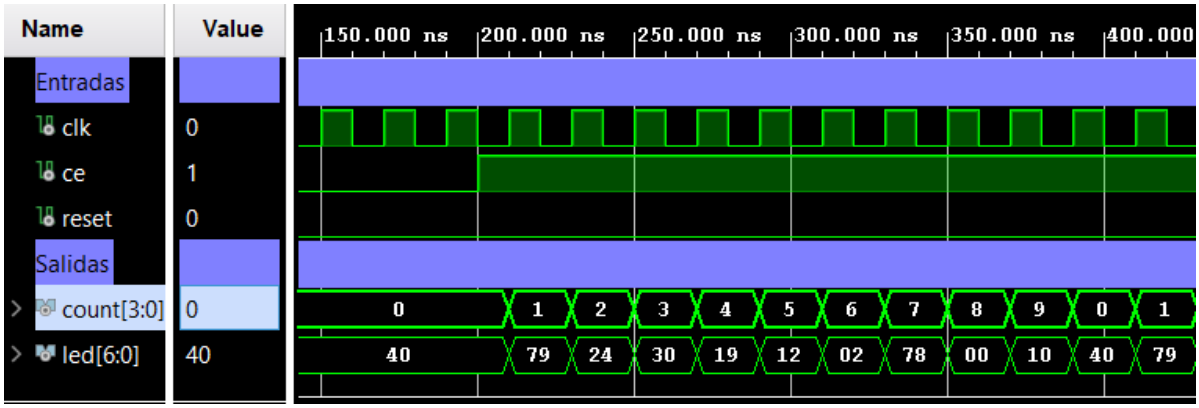


Figura 8. Captura de pantalla del cronograma de simulación del sistema.

5.- Asignación de pines de la FPGA

Para asignar qué pines de la FPGA se van a conectar a los puertos de entrada y salida del sistema hay que seguir el procedimiento especificado del Apartado 3 y siguientes de la Actividad 3. En este caso, las entradas de *reset* y *ce* se van a conectar a los pines de la FPGA que están conectados a los 2 interruptores más a la derecha de la tarjeta, la entrada de reloj se va a conectar al botón central de la tarjeta (de modo que fácilmente se puede generar una señal de reloj manual pulsando en él reiteradamente). En cuanto a los puertos de salida, la cuenta binaria *count* se asigna a los pines de la FPGA conectados a los 4 leds más a la derecha del array; y la salida correspondiente al decodificador, a los pines conectados a los segmentos de los displays de la tarjeta.

Así pues, de los periféricos de la tarjeta se usan 2 interruptores, un botón, 4 leds y 7 segmentos. El archivo de restricciones de usuario que queda usando la plantilla maestra se muestra en la Figura 12.

```
 9  set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets clk]
10
11  ##Switches
12  set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { ce }];
13  set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { reset }];
14
29  ## LEDs
30  set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { count[0] }];
31  set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { count[1] }];
32  set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { count[2] }];
33  set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { count[3] }];
34

55  ##7 segment display
56  set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { led[0] }];
57  set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { led[1] }];
58  set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { led[2] }];
59  set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { led[3] }];
60  set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { led[4] }];
61  set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { led[5] }];
62  set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { led[6] }];
63

76  ##Buttons
77  set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { clk }];
```

Figura 12. Captura de pantalla las líneas del archivo de restricciones correspondientes a los puertos del circuito *sistema.vhd*.

Es necesario incluir la directiva de la línea 9 para la señal de reloj *clk*. Esta directiva es necesaria para poder usar el pin de la FPGA N17 (línea 77) como señal de reloj y no una línea dedicada y diseñada para conducir este tipo de señales (consultar el Apartado 6 de la Actividad 5 para más información).

6.- Configuración del contador en la FPGA

El último paso consiste en configurar la tarjeta con el diseño del contador y haciendo uso de fichero de restricciones de usuario con los pines asignados a los puertos. Para realizar este paso hay que proceder de la misma manera que lo explicado en el Apartado 5 y el Apartado 6 de la Actividad 3. La Figura 13 muestra la tarjeta de desarrollo con el contador implementado en la FPGA Artix-7. **En este caso, el contador se encuentra habilitado (ce=1 y reset=0) y en la cuenta 5 (0101).**

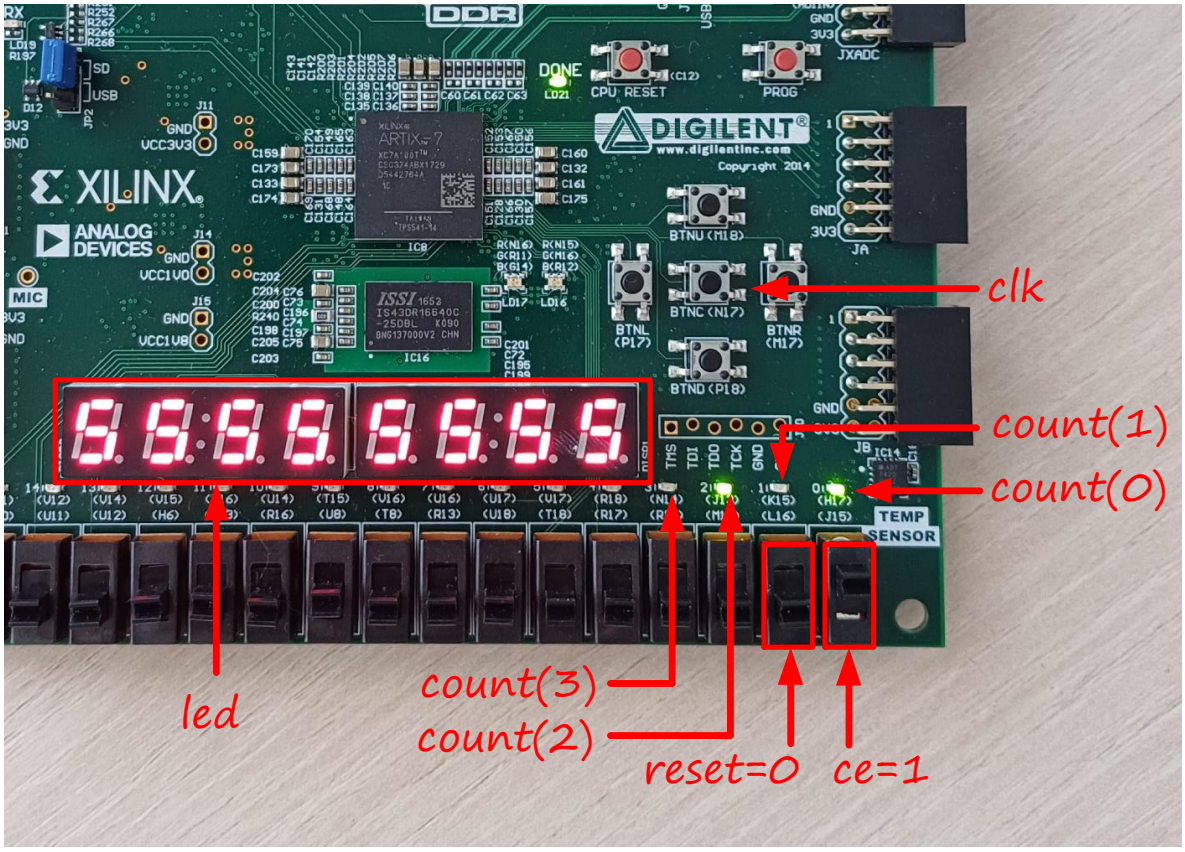


Figura 13. Foto de la tarjeta de desarrollo con el sistema implementado en la FPGA.

7.- Recursos de la FPGA empleados

Los recursos empleados para realizar este circuito se muestran en la Figura 14. En este diseño se emplean 14 pines de entrada/salida (correspondientes a los 3 puertos de entrada, *clk*, *ce* y *reset*; y los 11 puertos de salida, que son, los 4 de la salida binaria, *count*, y los 7 de la salida del

decodificador de 7 segmentos, *led*). Además, se emplean 4 flip-flops (para implementar el contador) y 7 LUT.

Utilization Post-Synthesis | **Post-Implementation**

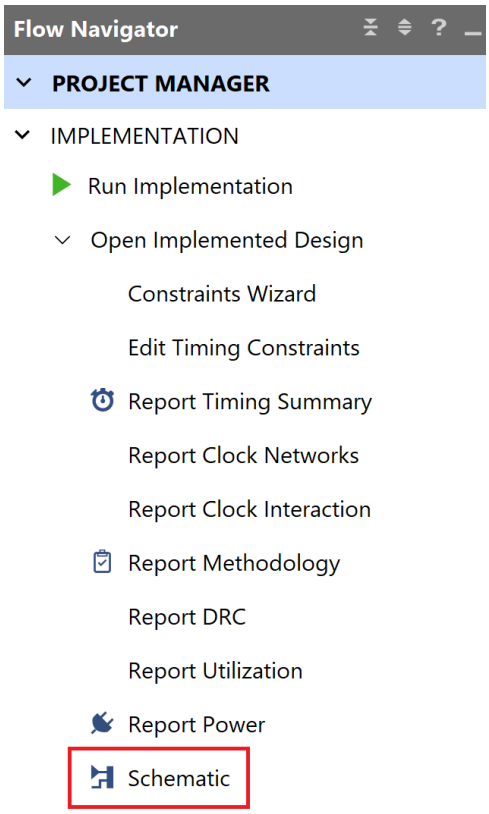
Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	7	63400	0.01
FF	4	126800	0.01
IO	14	210	6.67
BUFG	1	32	3.13

Figura 14. Captura de pantalla de la vista tabular de los recursos usados.

Se puede comparar los resultados de este diseño con el contador implementado en la Actividad 5. Los flip-flops empleados son los mismos, pero han sido necesario 4 puertos de salida adicionales (para la salida de la cuenta binario), así como 3 LUT adicionales.

8.- Esquemático estructural del diseño



En el entorno de desarrollo Vivado es posible visualizar un esquemático del diseño, justo al realizar la implementación en la fase de diseño del circuito. Para ello, en la ventana *Flow Navigator*, hay desplegar el apartado *Open Implemented Design* y pulsar en *Schematic* (Figura 15).

En la Figura 16 se muestra el esquemático del diseño, en el que puede apreciar los 3 puertos de entrada, los 11 puertos de salida, y los 2 subcircuitos que conforman el circuito total (*contador* y *dec7seg*).

Figura 15. Captura de pantalla del *Flow Navigator* con *Schematic* resaltado en rojo.

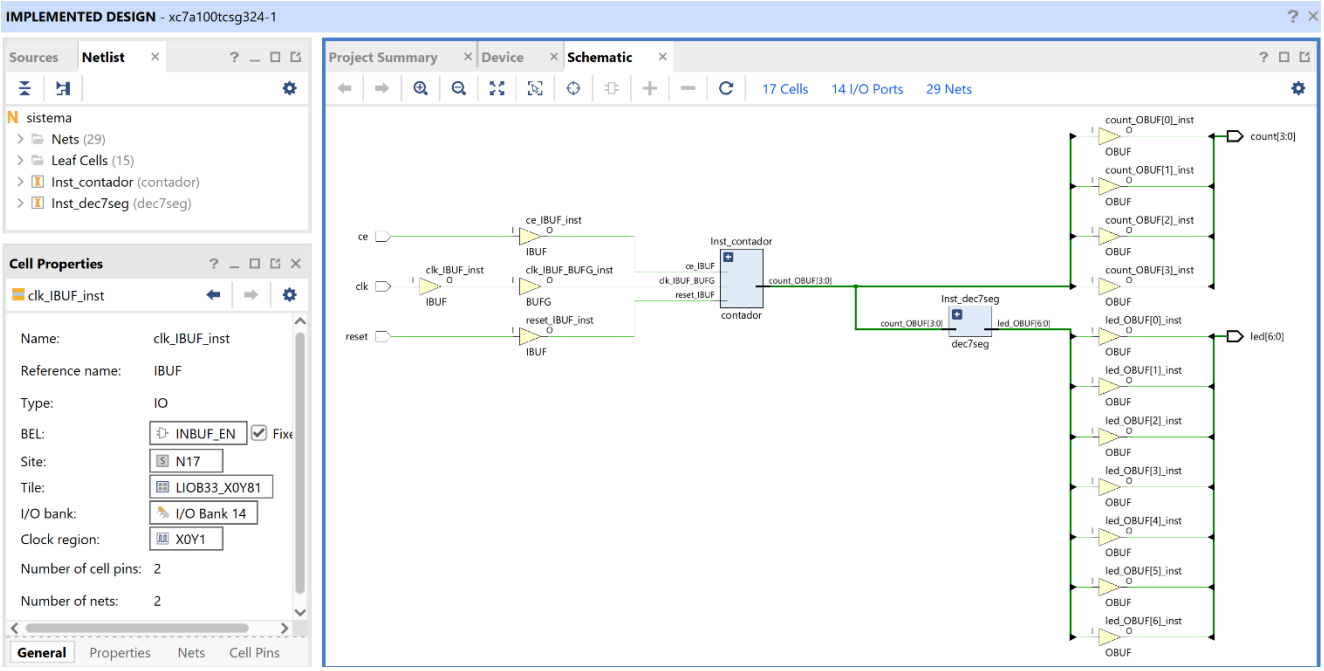


Figura 16. Captura de pantalla del esquemático del circuito.

A la izquierda del esquemática se muestran 2 ventanas: la ventana superior lista los componentes del esquemáticos (cables y celdas lógicas); y en la ventana inferior se muestra las propiedades del componente seleccionado en la ventana superior.

Es posible expandir en el esquemático cada subcircuito, para poder visualizarlo. Basta con pulsar en el signo "+" que aparece en la parte superior izquierda de cada uno de los dos bloques (Figura 17).

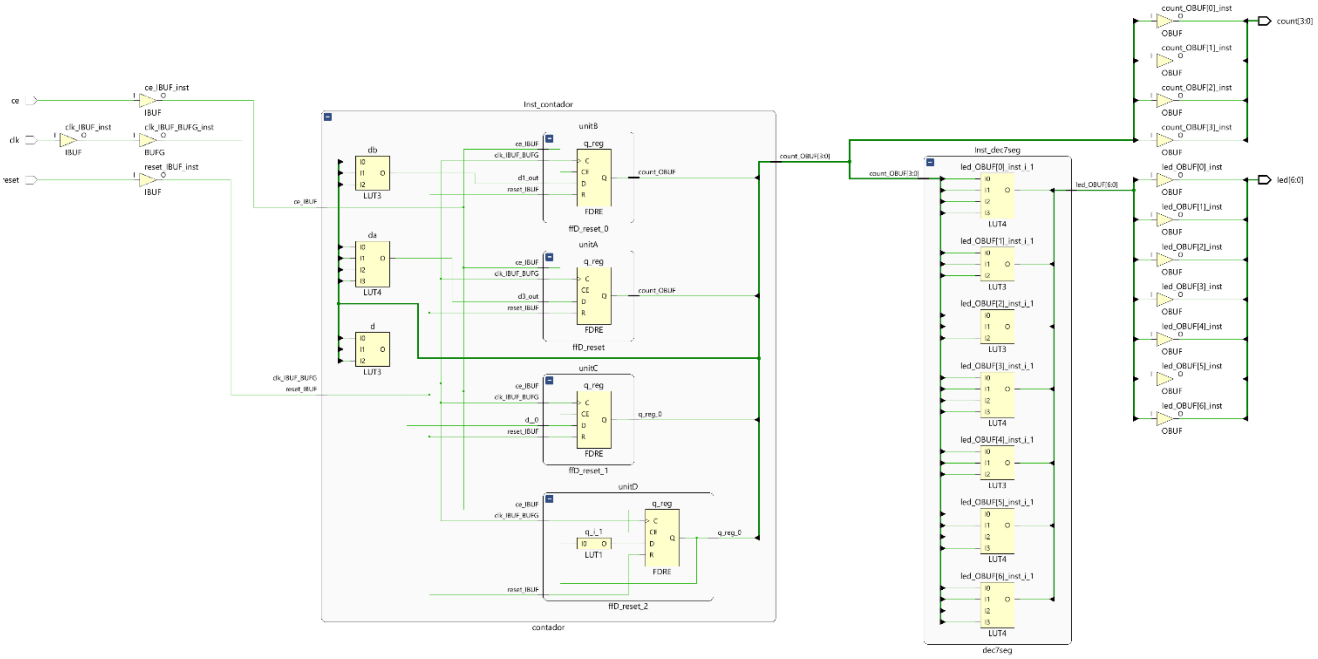


Figura 17. Captura de pantalla del esquemático del circuito.

Se puede comprobar que los LUT que se muestran en el esquemático NO coinciden con los recursos empleados que se muestran en la Figura 14. Esto se debe a que algunas de las LUT de la Figura 17 son LUT1, LUT3, LUT4, y las funciones lógicas que implementan se pueden agrupadas en las LUT6 de la Artix-7 (cada LUT6 puede inferir 2 funciones lógicas de hasta 5 variables cada una, confrontar con el Apartado 7 de la Actividad 5).

9.- Vista del circuito en la FPGA

En la ventana *Implemented Design* que se abre cuando visualizamos el esquemático en el Apartado 8, aparece una pestaña en la que se puede visualizar la propia FPGA en la que se esté realizando el diseño del circuito. Para ello, hay que hacer clic en la pestaña *Device* (Figura 18).

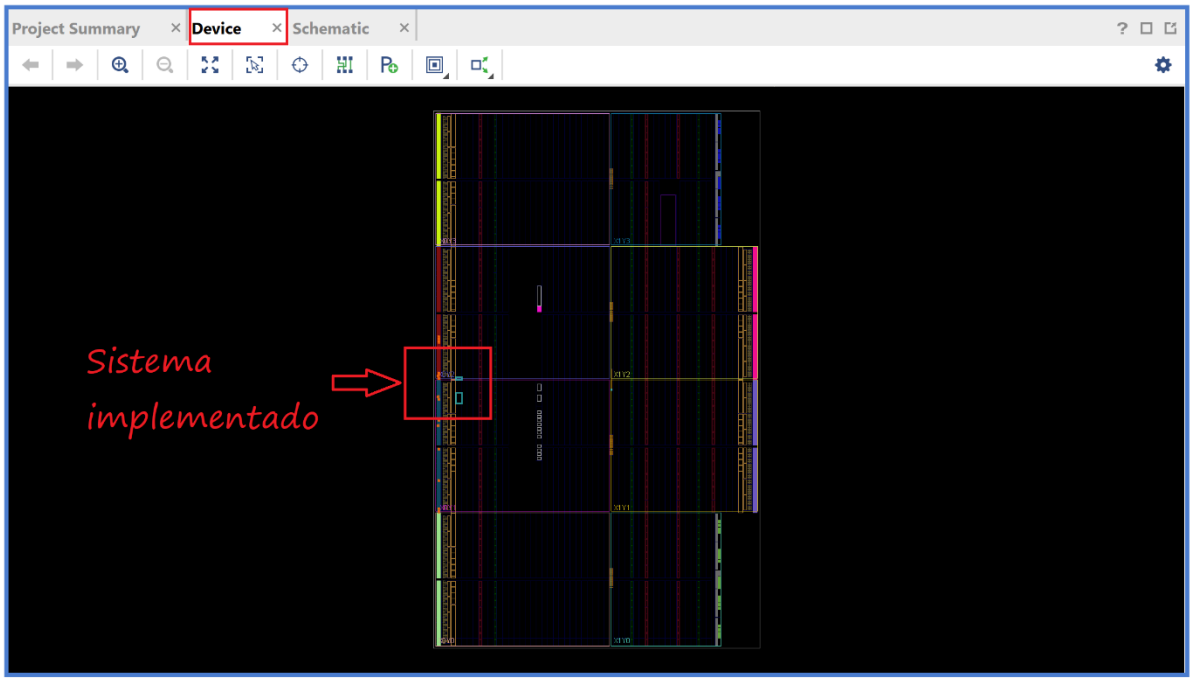


Figura 18. Captura de pantalla de la ventana *Device*, con la pestaña resaltada y la zona de la FPGA en la que se implementa el circuito señalada con una flecha.

En la vista interna de la FPGA que se muestra en la Figura 18 se aprecia en rectángulos azules los *slice* en lo que se ha mapeado el circuito descrito. Se puede hacer zoom sobre esta región para poder visualizar con mayor detalle la lógica inferida. Para ver las conexiones entre slice, hay que pulsar el icono .

Por ejemplo, en la Figura 19 se muestra el *slice* X0Y91 en que, en este caso en particular, se ha implementado 6 de las 7 funciones lógicas del decodificador de 7 segmentos, en concreto, todas las salidas excepto *led(2)*, como se puede observar en la Figura 20.

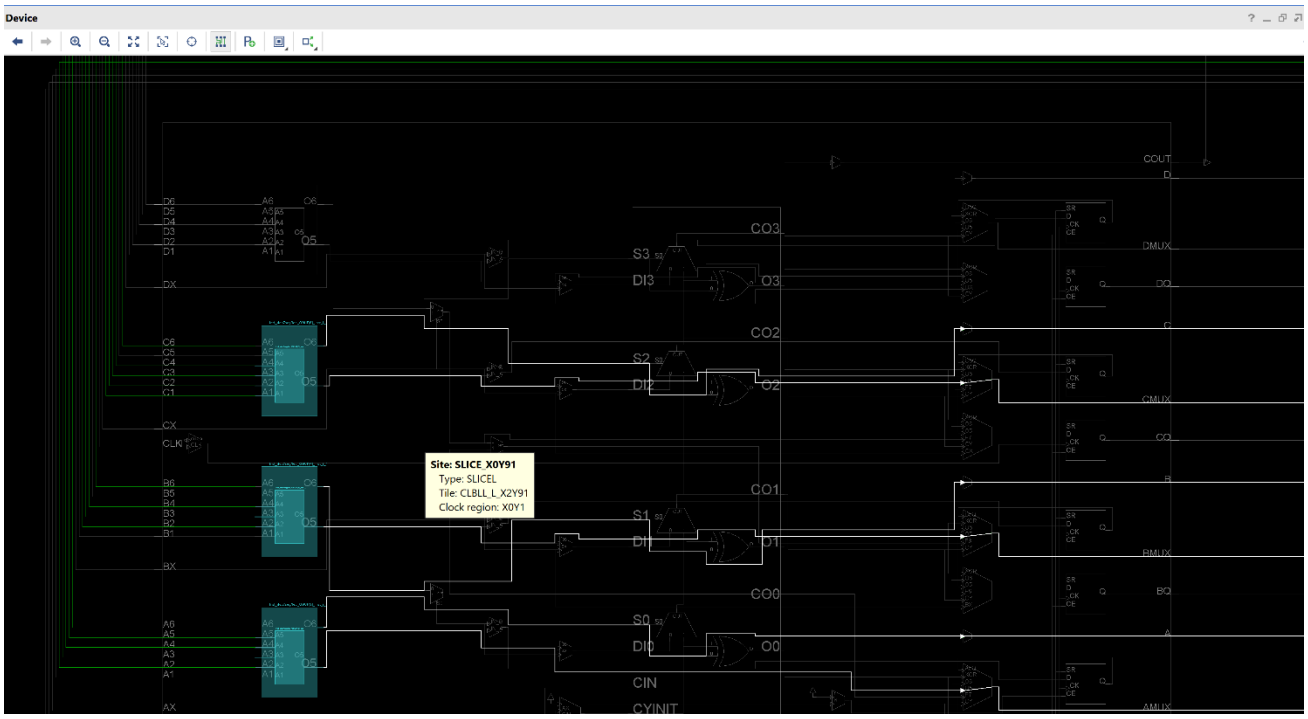


Figura 18. Captura de pantalla de la ventana *Device*, con el *slice* X0Y91 configurado.

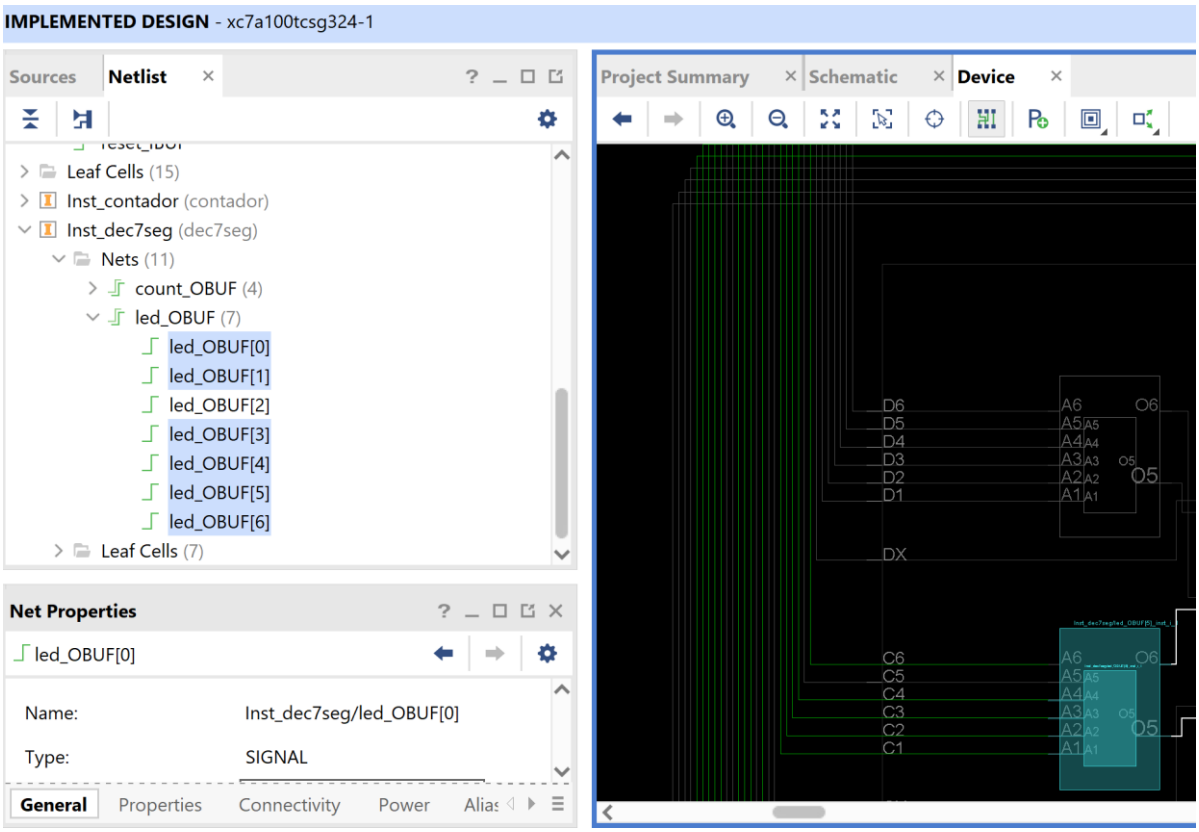


Figura 19. Captura de pantalla de la ventana *Device*, con el *slice* X0Y91 configurado.

En la Figura 20 se puede ver un detalle del *slice* X2Y93, del que se ha empleado 2 flip-flops para implementar los bits count(1) y count(3) de la

cuenta de 4 bits; así como una LUT6 para implementar las 2 funciones lógicas de entrada a estos dos flip-flops.

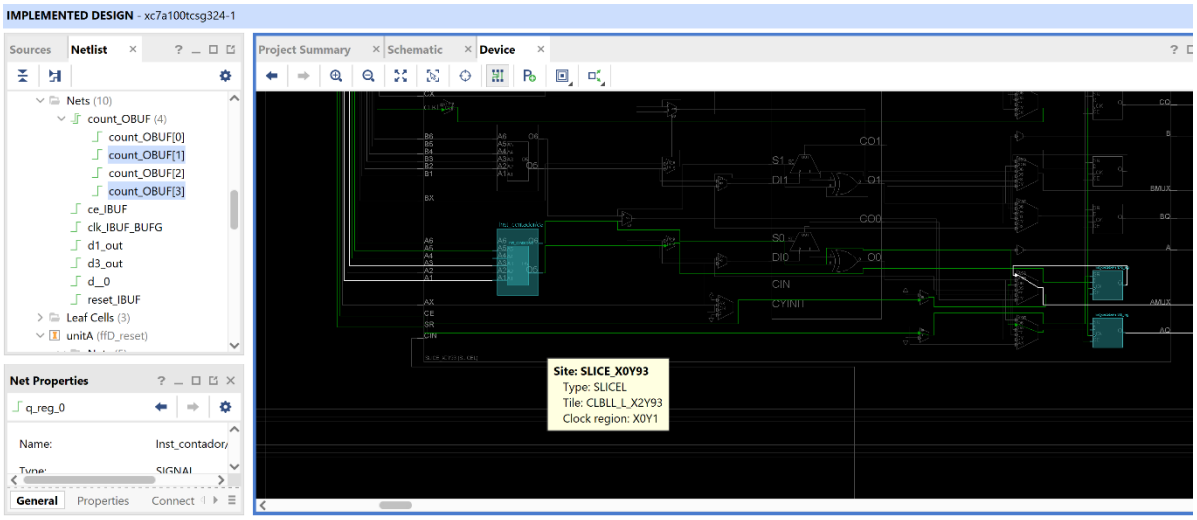


Figura 20. Captura de pantalla de la ventana *Device*, con el *slice* X2Y93 configurado.

Para cada LUT6 que se seleccione, se puede visualizar su configuración tanto en valor hexadecimal (Figura 21), como mostrando la tabla de verdad configurada (Figura 22).

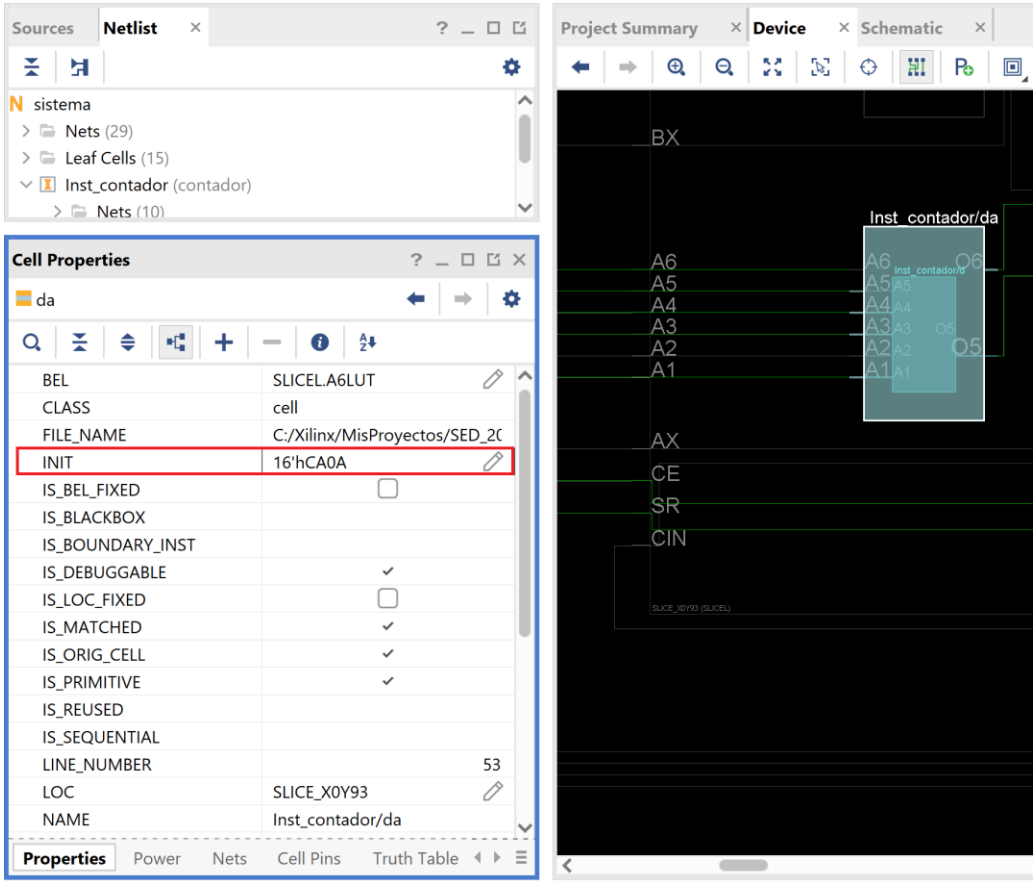


Figura 21. Captura de pantalla de las propiedades de la LUT6 con *da* implementado. Su tabla de verdad es CA0A.

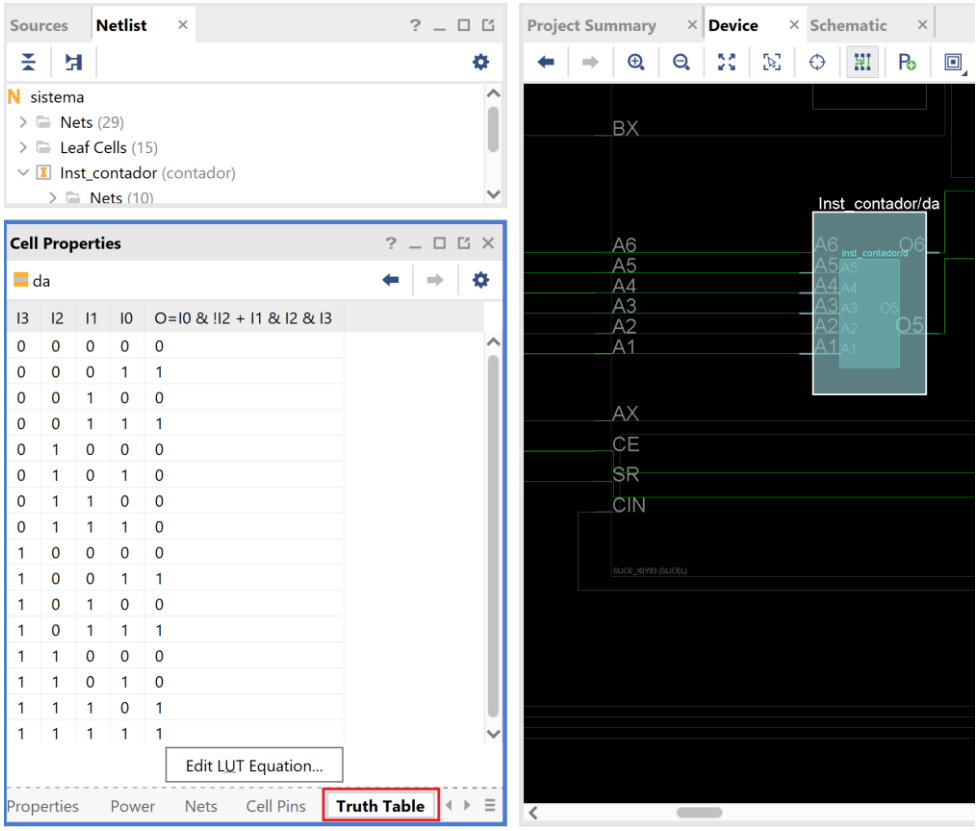


Figura 22. Captura de pantalla de las propiedades de la LUT6 con *da* implementado. Su tabla de verdad es CA0A.

Es importante tener en cuenta que las tablas de verdad que se infieren normalmente NO coinciden con las del diseñador, ya que Vivado suele escoger orden de peso de las variables de entrada. En la Figura 23 se muestra que, en este caso, para la implementación de la función *da*, se escoge *da(q1,q0,q2,q3)* en lugar de *da(q3,q2,q1,q0)*, que es como se obtuvo en la Actividad 5. Se trata de la misma función, pero la tabla de verdad tiene otro orden.

Si se realiza la descripción del diseño a nivel estructural, la información del orden de las variables se le aporta a Vivado en los PORT MAP, así que, en este caso, la tabla de verdad sí coincide (ver Apartado 9 de la Actividad 4).

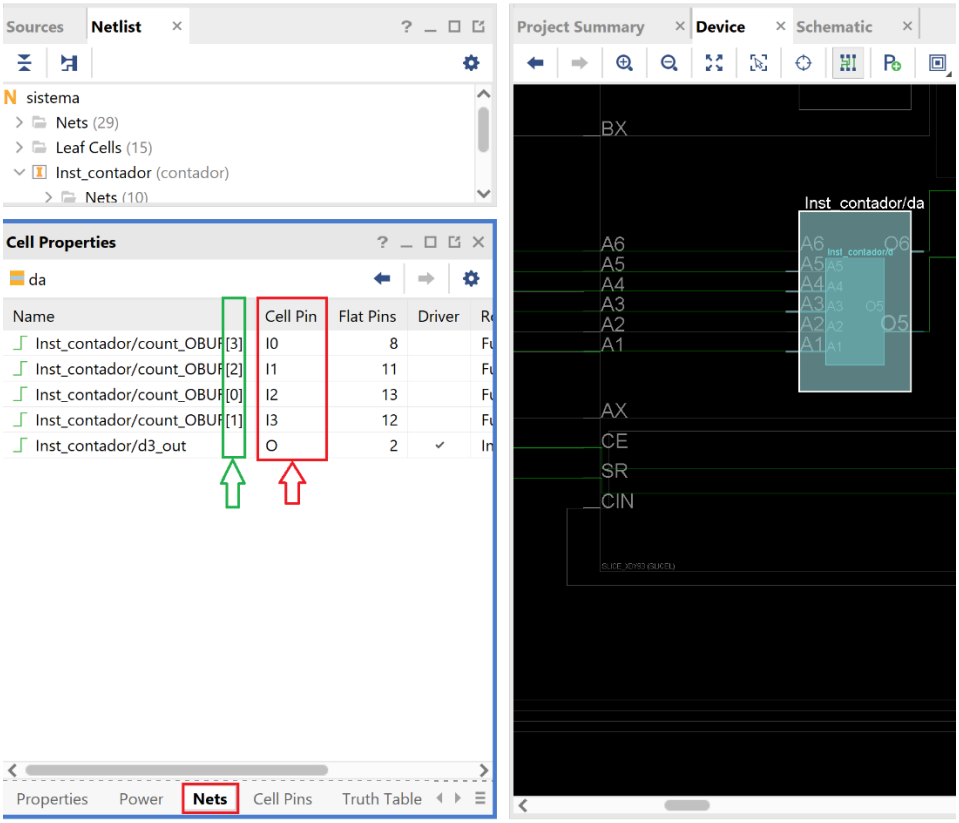


Figura 22. Captura de pantalla de las propiedades de la LUT6 con la pestaña *Nets* seleccionada.

Referencias

[1] https://docs.xilinx.com/v/u/en-US/ug474_7Series_CLB
[2] https://docs.xilinx.com/r/en-US/ug953-vivado-7series-libraries